

Punto Medio

Lang Lovdog

Resolución de punto medio de dos gaussianas

Aclaración:

Estas notas son solo para términos interesantes y detalles programáticos. Pero no corresponde al método de resolución del problema per se.

En el detalle de MATLAB, las notas están en papel. Las transcribiré con tiempo.

Método en C

Se puede definir una estructura de polinomio para este caso en particular.

```
typedef struct poly{  
    double *a; // Coeficientes del polinomio en cuestión.  
    size_t n; // Grado del polinomio  
}
```

Esta estructura requiere algunas operaciones importantes:

Creación e impresión en consola

Esta parte no la menciona como tal en la clase, pero es necesario desde mi perspectiva, tener una función correcta de creación de la estructura.

```
char crea_poly(poly *fx, size_t grado){  
    (*fx).a = (float*)calloc(grado+1,sizeof(float));  
    (*fx).n = grado;  
}  
char imprimir_poly(poly fx){  
    size_t i;  
    i=fx.n; while((i--)>1) printf("%+lfx^%lu",fx.a[i],i);  
    printf("\n");  
}
```

Evaluación

Este proceso hace uso del multiplicador-acumulador, lo cual permite una sola operación en bajo nivel para la evaluación de cada nivel del polinomio.

```
double evaluar(poly fx, double x){
    size_t i;
    double r;
    i=0; r=fx.a[n]; while(i<fx.n) r = r*x+fx.a[fx.n-(i++)+1];
}
```

Derivación

```
poly derivar(poly *fx){
    poly dfx;
    size_t i;
    dfx.n = (*fx).n-1;
    crea_poly(&dfx,dfx.n);
    i=0; while((i++)<dfx.n) dfx.a[i-1] = (i) * (*fx).a[i]
}
```

Resolución (raíces del polinomio)

Método de bisección

```
double raiz_poly_bisecccion(poly fx, double x1, double x2, double tol){
    double fx1, fx2, xr, fxr, err;
    fx1=evaluar(fx,x1);
    fx2=evaluar(fx,x2);
    if (fx1*fx2>0) return 0;
    while(err>tol){
        if (fxr*fx2<0){ x1=xr; fx1=fxr; fabs((xr-x1)/xr)} else
        if (fx1*fxr<0){ x2=xr; fx2=fxr; fabs((xr-x2)/xr)} else
        return xr;
    }
}
```

Durante esta práctica hubo un detalle importante. Éste método es bastante sensible a los valores del rango de búsqueda.

Nota del día de hoy

Esto es un planteamiento matemático algo extenso

Tenemos una función $f(x)$ que tiene ciertas raíces $x_1, x_2, \dots$, se calcula el error cuadrático de la forma $\sum_{i=1}^n (f(x_i) - f(x_i))^2$

Como punto de partida, lo más importante es poder obtener un sistema de ecuaciones, la forma de crearlo es partiendo de derivadas parciales con respecto a cada coeficiente.

Al ser valores de error, teóricamente cada uno estará igualado a 0, lo que nos permite crear un sistema de ecuaciones simple.

Después de aplicar algunos artificios matemáticos, podremos obtener una combinación lineal de la forma $Ax = b$ con ecuaciones de dimensión n (de orden $n - 1$).

$$\begin{bmatrix} n & \sum x_i & \cdots & \sum x_i^{(n-1)} \\ \sum x_i & \sum x_i^2 & \cdots & \sum x_i^n \\ \vdots & \vdots & \ddots & \vdots \\ \sum x_i^{(n-1)} & \sum x_i^3 & \cdots & \sum x_i^{2(n-1)} \end{bmatrix}$$

Solución por Gauss-Jordan

Supongamos una matriz que representa la combinación lineal de la forma $Ax = b$. Dicha matriz se puede expresar como:

$$\begin{bmatrix} a_{1\ 1} & a_{1\ 2} & \cdots & a_{1\ n} \\ a_{2\ 1} & a_{2\ 2} & \cdots & a_{2\ n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n\ 1} & a_{n\ 2} & \cdots & a_{n\ n} \end{bmatrix}$$

Cada iteración dentro de la matriz permite ser definido a través de operaciones que se sustentan en un patrón.

$$\begin{bmatrix} a_{1\ 1} & a_{1\ 2} & \cdots & a_{1\ n} \\ 0 & a_{2\ 2} - \frac{a_{2\ 1}a_{1\ 2}}{a_{1\ 1}} & \cdots & a_{2\ n} - \frac{a_{2\ 1}a_{1\ n}}{a_{1\ 1}} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n\ 2} & \cdots & a_{n\ n} \end{bmatrix} \begin{bmatrix} a_{1\ 1} & a_{1\ 2} & \cdots & a_{1\ n} \\ 0 & a_{2\ 2} - \frac{a_{2\ 1}a_{1\ 2}}{a_{1\ 1}} & \cdots & a_{2\ n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n\ 2} & \cdots & a_{n\ n} \end{bmatrix}$$

*/*Este algoritmo propone el método Gauss-Jordan, «recorriendo» una matriz que supone una solución general d'un sistema de ecuaciones.*/*

```
for(int i=1; i<n; ++i) // índice pivote de la matriz
    for(int j=i; i<n; ++j) // índice del renglón
        for(int k=1; i<n; ++k) // índice de la columna
```